```
object DataEnricher {
  sealed trait Command
  final case class Message(id: String, content: String) extends Command

  def apply() = Behaviors.setup[Command] { context =>
    val router = ...

    Behaviors.receiveMessage[Command] {
      case Message(id, content) =>
        router ! Aggregator.Enriched(id, content + ":metadata")
        Behaviors.same
    }
}}
```

The router uses the ID that comes from the original message to calculate the hash. The same is true for the router from the obfuscator, so the aggregator receives both processing parts of the same message.

In this section, you learned how to use Akka routers, which are used for performance reasons, but remember that routers are also used based on the content of a message or the state of the actor. The next section examines the content- and state-based routing approaches.

6.4 Implementing the router pattern using actors

To route in Akka, you are not always required to implement a pool or group router. If the router’s decision is based on the message or the state of the actor, it is easier to implement it in a normal actor with the behaviors API.

This section looks at some ideas for the router pattern with normal actors. We start with a message-based router and follow it with a